Отчёт по лабораторной работе №1

Дисциплина: Компьютерный практикум по статистическому анализу данных

Выполнил: Танрибергенов Эльдар (НПИбд-01-22)

Содержание

1 Цель работы **6**

2 Выполнение лабораторной работы **7**

2.1 Подготовка инструментария к работе 7

2.2 Основы работы в блокноте Jupyter 8

2.3 Основы синтаксиса Julia на примерах 12

2.4 Задания для самостоятельной работы 14

3 Выводы **26**

Список иллюстраций

2.1 Установка Julia 7
2.2 Установка необходимого ПО 7
2.3 Установка Jupyter 8
2.4 Запуск Julia и установка пакетов для работы с Jupyter 8
2.5 Запуск Jupyter Lab 9
2.6 Простейшие операции на языке Julia в Jupyter Lab 10
2.7 Пример получения информации по функции println на языке Julia в Jupyter Lab 11
2.8 Пример получения информации о дате и пользователе ОС Linux в Jupyter Lab 11
2.9 Примеры определения типа числовых величин 12
2.10 Примеры приведения аргументов к одному типу 12
2.11 Примеры определения функций 13
2.12 Примеры работы с массивами 14
2.13 Текстовый файл для теста 15
2.14 Пример использования функции read() 15
2.15 Пример использования функции readline() 16
2.16 Пример использования функции readlines() 16
2.17 Пример использования функции print() 16
2.18 Пример использования функции println() 17
2.19 Пример использования функции show() 17
2.20 Пример использования функции write() 17
2.21 Проверка изменений в файле 18
2.22 Примеры использования функции parse() 18
2.23 Пример использования операции сложения 19
2.24 Пример использования операции вычитания 19
2.25 Пример использования операции умножения 20
2.26 Пример использования операции деления 20
2.27 Пример использования операции возведения в степень 21
2.28 Пример использования операции извлечения корня 21
2.29 Пример использования операции сравнения 22
2.30 Особенность сравнения вещественных чисел 22
2.31 Пример использования логических операций 22
2.32 Пример сложения матриц 23
2.33 Пример сложения векторов 23
2.34 Пример вычитания матриц 23
2.35 Пример вычитания векторов 24

2.36 Пример скалярного произведения векторов 24
2.37 Пример транспонирования матрицы 24
2.38 Пример транспонирования вектора 24
2.39 Пример умножения вектора на скаляр 25
2.40 Пример умножения матрицы на скаляр 25

Список таблиц

1 Цель работы

Основная цель работы — подготовить рабочее пространство и инструментарий для работы с языком программирования Julia, на простейших примерах познакомиться с основами синтаксиса Julia.

2 Выполнение лабораторной работы

2.1 Подготовка инструментария к работе

Установил Julia (https://julialang.org/) и Jupyter (https://jupyter.org/) под операционную систему Windows посредством менеджера пакетов Chocolatey (https://chocolatey.org/). Далее установил Far Manager, Notepad++, Julia, Anaconda Distribution (Python 3.x), Jupyter.

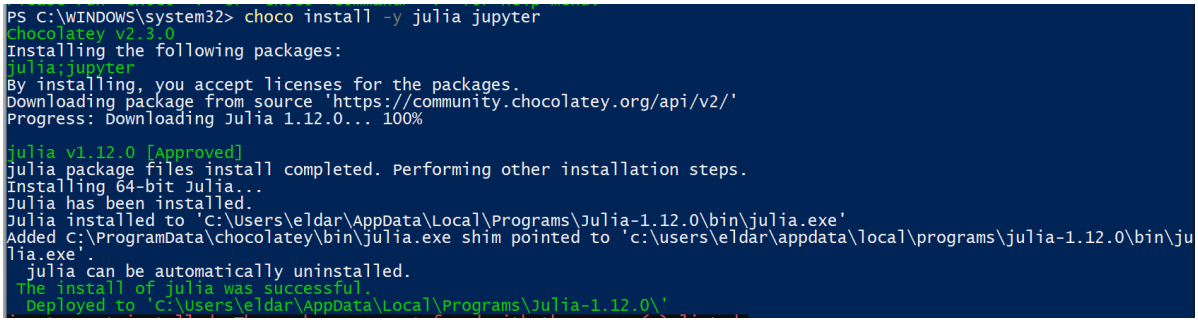

Рис. 2.1: Установка Julia

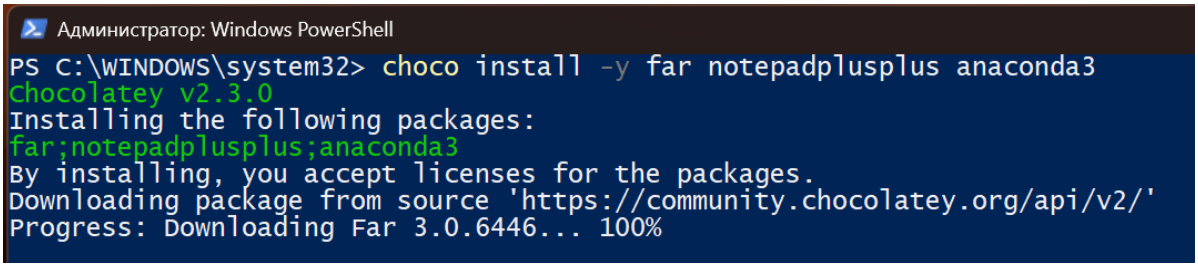

Рис. 2.2: Установка необходимого ПО

```
PS C:\WINDOWS\system32> pip install jupyter
Collecting jupyter
  Downloading jupyter-1.1.1-py2.py3-none-any.whl.metadata (2.0 kB)
```

Рис. 2.3: Установка Jupyter

После установки запустил Julia в режиме REPL (read-eval-print loop). После запуска Julia попал в режим командной строки. Установил пакеты для работы с Jupyter. Для этого перешёл в пакетный режим Julia, нажав на клавиатуре знак закрывающейся квадратной скобки `]`, затем ввёл `add IJulia`.

```
Julia 1.12.0

               _
   _       _ _(_)_     |  Documentation: https://docs.julialang.org
  (_)     | (_) (_)    |
   _ _   _| |_  __ _   |  Type "?" for help, "]?" for Pkg help.
  | | | | | | |/ _` |  |
  | | |_| | | | (_| |  |  Version 1.12.0 (2025-10-07)
 _/ |\__'_|_|_|\__'_|  |  Official https://julialang.org release
|__/                   |

(@v1.12) pkg> add IJulia
  Installing known registries into `C:\Users\eldar\.julia`
      Added `General` registry to C:\Users\eldar\.julia\registries
    Updating registry at `C:\Users\eldar\.julia\registries\General.toml`
   Resolving package versions...
```

Рис. 2.4: Запуск Julia и установка пакетов для работы с Jupyter

Для интерактивной работы с Julia удобно использовать или Jupyter Notebook, или Jupyter Lab. По своей сути блокнот Jupyter позволяет объединить в единый документ тест, программный код, результат его выполнения и визуализацию результата. Основная работа с блокнотами осуществляется посредством браузера. Формат документов (.ipynb) идентичен в Jupyter Notebook и Jupyter Lab.

2.2 Основы работы в блокноте Jupyter

Запустил Jupyter Lab.

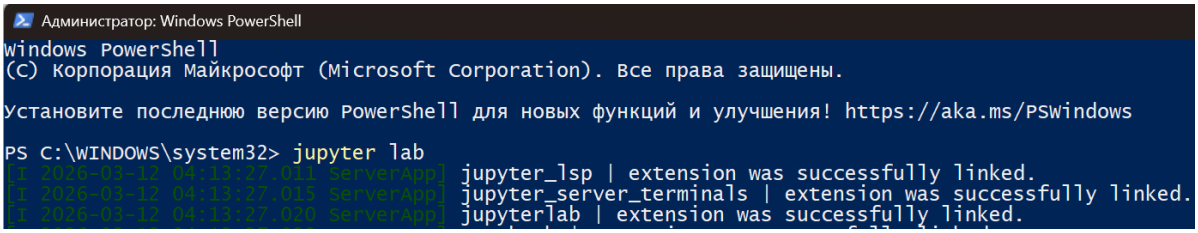

Рис. 2.5: Запуск Jupyter Lab

Каждый блокнот (или консоль, или терминал, или текстовый редактор) располагается в своей вкладке в основной рабочей области. Для создания нового блокнота выбрал в меню File->New , далее указал, что именно создать - Notebook , затем ядро Juliia-1.x . Основная концепция интерактивных блокнотов — это ячейка, содержащая отдельный фрагмент текста (или кода). Для написания текста в ячейке нужно в панели инструментов указать Markdown, для написания элемента кода — Code. Для изменения режимов вставки ячеек можно использовать также комбинации клавиш. Для этого нужно на активной ячейке нажать ESC , что выведет ячейку из режима редактирования и переведёт её в командный режим, в котором есть специальные сочетания клавиш для вставки / вырезания / изменения ячеек: – a или b — создать новую ячейку соответственно выше или ниже текущей; – x — удалить ячейку; – z — отмена удаления ячейки; – m — перевести ячейку в режим текста; – y — перевести ячейку в режим набора кода. Для выполнения кода внутри ячейки выберите эту ячейку и нажмите Shift + Enter или кнопку со значком Run на панели инструментов. Если ячейка содержит несколько строк кода, то при выполнении этой ячейки отобразится только результат последней строки (операции). Вывод результата можно подавить, завершив строку знаком «точка с запятой».

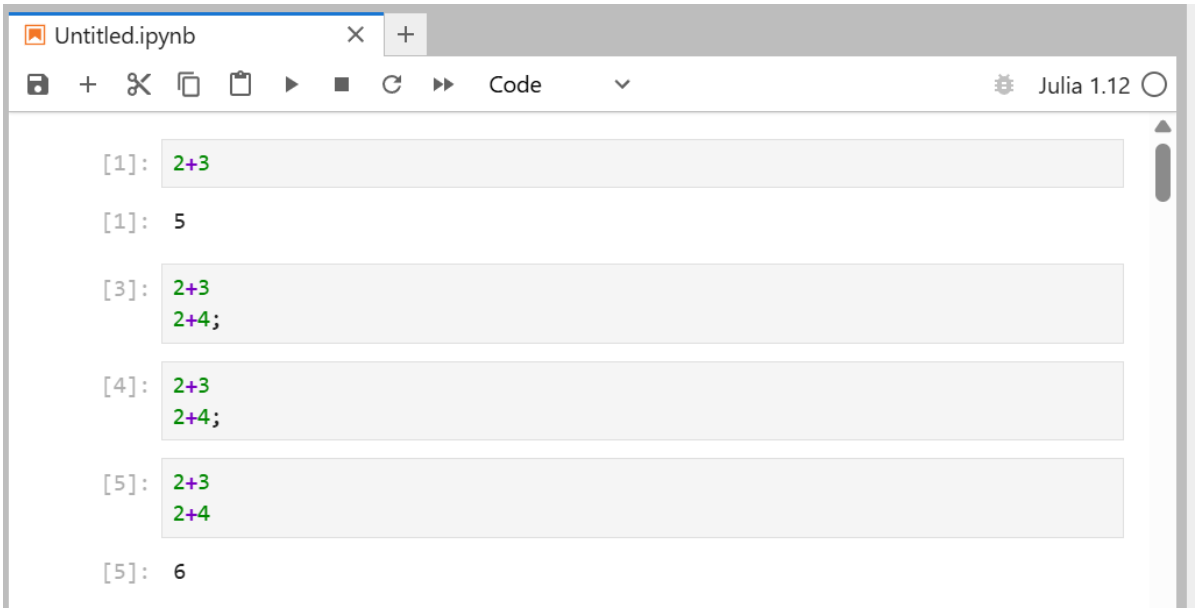

Рис. 2.6: Простейшие операции на языке Julia в Jupyter Lab

Если необходимо получить информацию по работе с какой-то незнакомой для вас функцией Julia, то надо поставить в ячейке перед названием этой функции знак вопроса

```
[6]: ?println
```

```
search: println print sprint pointer printstyled
```

[6]:

```
    println([io::IO], xs...)
```

Print (using `print`) `xs` to `io` followed by a newline. If `io` is not supplied, prints to the default output stream `stdout`.

See also `printstyled` to add colors etc.

Examples

```
julia> println("Hello, world")
Hello, world

julia> io = IOBuffer();

julia> println(io, "Hello", ',', " world.")

julia> String(take!(io))
"Hello, world.\n"
```

Рис. 2.7: Пример получения информации по функции println на языке Julia в Jupyter Lab

Если требуется использовать команды из командной оболочки вашей операционной системы, то перед соответствующей командой нужно поставить знак «точка с запятой».

```
[1]: ;date
```

```
Thu Mar 12 04:47:27 RTZ 2026
```

```
[2]: ;whoami
```

```
eldar\eldar
```

Рис. 2.8: Пример получения информации о дате и пользователе ОС Linux в Jupyter Lab

2.3 Основы синтаксиса Julia на примерах

Далее приведены простейшие примеры с использованием синтаксиса Julia, выполненные в блокноте Jupyter Lab. Определил крайних значений диапазонов целочисленных числовых величин:

```
eldar\eldar

[3]: for T in
         [Int8,Int16,Int32,Int64,Int128,UInt8,UInt16,UInt32,UInt64,UInt128]
       println("$(lpad(T,7)): [$(typemin(T)),$(typemax(T))]")
     end

   Int8: [-128,127]
  Int16: [-32768,32767]
  Int32: [-2147483648,2147483647]
  Int64: [-9223372036854775808,9223372036854775807]
 Int128: [-170141183460469231731687303715884105728,170141183460469231731687303715884105727]
  UInt8: [0,255]
 UInt16: [0,65535]
 UInt32: [0,4294967295]
 UInt64: [0,18446744073709551615]
UInt128: [0,340282366920938463463374607431768211455]
```

Рис. 2.9: Примеры определения типа числовых величин

Попробовал преобразование типов:

```
[4]: Int64(2.0), Char(2), typeof(Char(2))

[4]: (2, '\x02', Char)

[5]: convert(Int64, 2.0), convert(Char,2)

[5]: (2, '\x02')

[6]: typeof(promote(Int8(1), Float16(4.5), Float32(4.1)))

[6]: Tuple{Float32, Float32, Float32}
```

Рис. 2.10: Примеры приведения аргументов к одному типу

Базовый синтаксис определения функции:

```
function <Имя> (<СписокПараметров>)
    <Действия>
end
```

Другой способ определения несложных функций:

```
<Имя> (<СписокПараметров>) = <Выражение>
```

Например, определим функцию $f(x)$ возведения переменной x в квадрат и возведём

```
[7]: function f(x)
         x^2
     end
     f(4)

[7]: 16

[8]: g(x)=x^2

[8]: g (generic function with 1 method)

[9]: g(8)

[9]: 64
```

Рис. 2.11: Примеры определения функций

Работа с массивами:

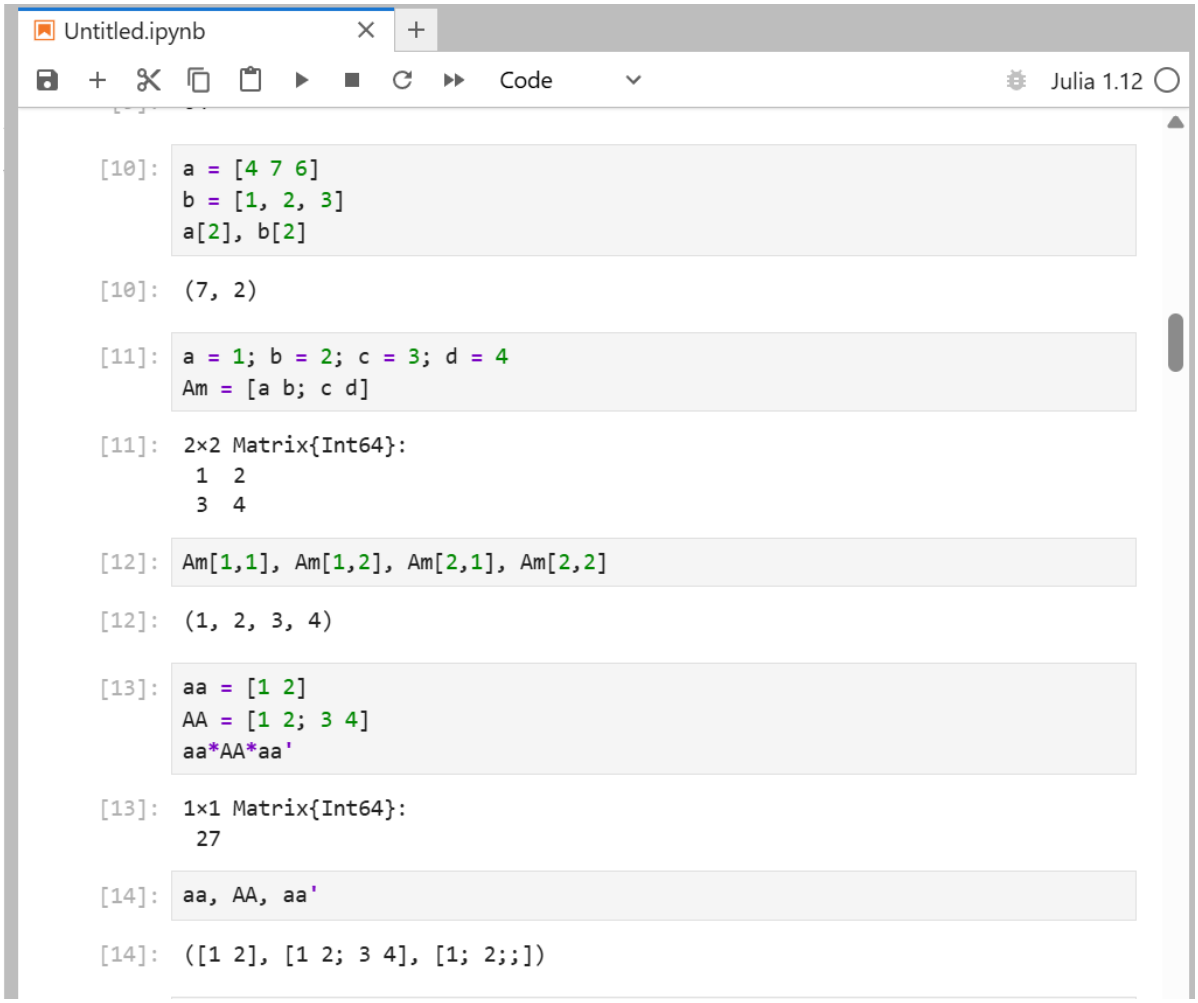

Рис. 2.12: Примеры работы с массивами

2.4 Задания для самостоятельной работы

1. Изучил документацию по основным функциям Julia для чтения / записи / вывода информации на экран: read(), readline(), readlines(), print(), println(), show(), write().

Примеры использования функций:

- **read()**

Для этого создал текстовый файл:

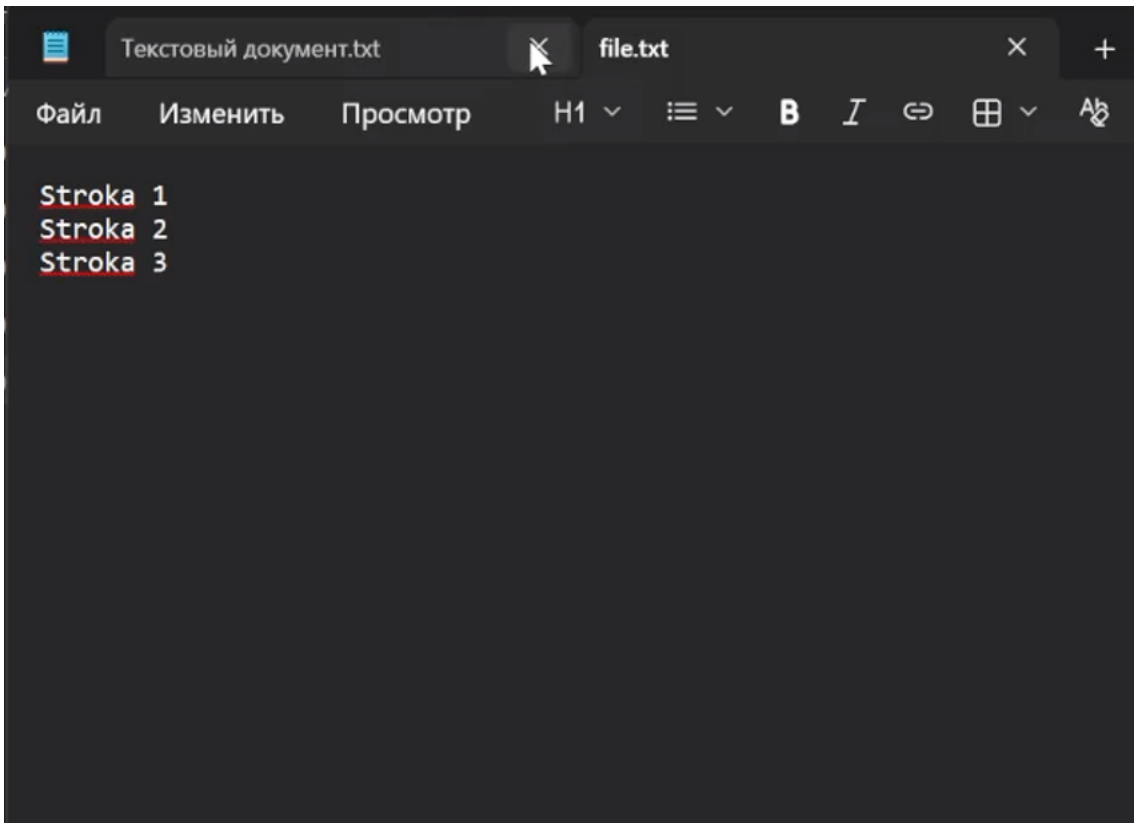

Рис. 2.13: Текстовый файл для теста

Функция read() читает данные из потока в заданном формате.

```
[25]: content = read("C:\\work\\study\\2025-2026\\file.txt", String)
      println(content)

      Stroka 1
      Stroka 2
      Stroka 3
```

Рис. 2.14: Пример использования функции read()

- **readline()**

Читает одну строку из файла или потока до символа новой строки.

```
[27]: open("C:\\work\\study\\2025-2026\\file.txt") do file
          line1 = readline(file)
          line2 = readline(file)
          println("Первая строка: $line1")
          println("Вторая строка: $line2")
      end

      Первая строка: Stroka 1
      Вторая строка: Stroka 2
```

Рис. 2.15: Пример использования функции readline()

- **readlines()**

Читает все строки файла и возвращает их в виде массива.

```
[28]: lines = readlines("C:\\work\\study\\2025-2026\\file.txt")
      for (i, line) in enumerate(lines)
          println("Строка $i: $line")
      end

      Строка 1: Stroka 1
      Строка 2: Stroka 2
      Строка 3: Stroka 3
```

Рис. 2.16: Пример использования функции readlines()

- **print()**

Выводит данные в стандартный поток вывода (консоль) без добавления символа новой строки.

```
[29]: print("на ")
      print("одной ")
      print("строке")

      на одной строке
```

Рис. 2.17: Пример использования функции print()

- **println()**

Аналогично print(), но добавляет символ новой строки в конец.

```
[32]: println("на ")
      println("разных ")
      println("строках")

      на
      разных
      строках
```

Рис. 2.18: Пример использования функции println()

- **show()**

Выводит внутреннее представление объекта, полезное для отладки.

```
[65]: x = [1, 2, 3]
      show(stdout, x)

      [1, 2, 3]
```

Рис. 2.19: Пример использования функции show()

- **write()**

Записывает данные в файл или поток.

```
[35]: open("C:\\work\\study\\2025-2026\\file.txt", "w") do file
          write(file, "Новая строка 1\n")
          write(file, "Новая строка 2\n")
      end
```

Рис. 2.20: Пример использования функции write()

Проверил файл:

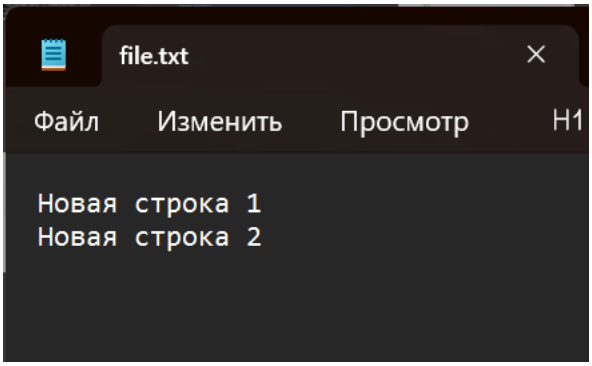

Рис. 2.21: Проверка изменений в файле

2. Изучил документацию по функции parse() и опробовал.

Функция parse() преобразует строку в значение указанного типа.

```
[40]: parse(Int, "101", base = 2)

[40]: 5

[41]: print("Введите число: ")
      input = readline()
      try
          number = parse(Float64, input)
          println("Вы ввели: $number")
      catch
          println("Это не число!")
      end

      Введите число:
      stdin>  14
      Вы ввели: 14.0
```

Рис. 2.22: Примеры использования функции parse()

3. Изучил синтаксис Julia для базовых математических операций с разным типом переменных: сложение, вычитание, умножение, деление, возведение в степень, извлечение корня, сравнение, логические операции.

Примеры использования функций:

- **Сложение**

```
[43]: a = 4 + 1
      b = 1.5 + 6.7
      c = (1 + 2im) + (3 + 4im)
      d = 14 + 1.4
      println(a)
      println(b)
      println(c)
      println(d)

      5
      8.2
      4 + 6im
      15.4
```

Рис. 2.23: Пример использования операции сложения

- **Вычитание**

```
[44]: x = 14 - 4
      y = 6.5 - 2.1
      z = (4 + 3im) - (1 + 2im)
      println(x)
      println(y)
      println(z)

      10
      4.4
      3 + 1im
```

Рис. 2.24: Пример использования операции вычитания

- **Умножение**

Julia поддерживает неявное умножение: ax эквивалентно a * x

```
[46]: a = 4 * 6
      b = 2.5 * 3.0
      c = (2 + 3im) * (1 + 1im)
      d = 2(3 + 4)
      e = 3x
      println(a)
      println(b)
      println(c)
      println(d)
      println(e)

      24
      7.5
      -1 + 5im
      14
      30
```

Рис. 2.25: Пример использования операции умножения

- **Деление**

Julia поддерживает два вида деления: - Обычное деление (/) - Целочисленное деление (÷ или div())

```
[47]: a = 10 / 2
      b = 7 / 3
      c = 10 ÷ 3
      d = div(15, 4)
      println(a)
      println(b)
      println(c)
      println(d)

      5.0
      2.3333333333333335
      3
      3
```

Рис. 2.26: Пример использования операции деления

- **Возведение в степень**

```
[48]: a = 2 ^ 3
      b = 4 ^ 0.5
      c = 2 ^ -2
      d = (1 + im) ^ 2
      println(a)
      println(b)
      println(c)
      println(d)

      8
      2.0
      0.25
      0 + 2im
```

Рис. 2.27: Пример использования операции возведения в степень

- **Извлечение корня**

Julia поддерживает функции извлечения квадратного и кубического корней, а также извлечения корня произвольной степени путём возведения в степень.

```
[50]: a = sqrt(9)
      b = cbrt(27)
      c = 9 ^ (1/2)
      d = 27 ^ (1/3)
      println(a)
      println(b)
      println(c)
      println(d)

      3.0
      3.0
      3.0
      3.0
```

Рис. 2.28: Пример использования операции извлечения корня

- **Сравнение**

```
[51]: x = 1
      y = 2

      println(x == y)
      println(x != y)
      println(x > y)
      println(x < y)
      println(x >= y)
      println(x <= y)

      false
      true
      false
      true
      false
      true
```

Рис. 2.29: Пример использования операции сравнения

Из-за погрешности вычислений чисел с плавающей точкой, существует следующее:

```
[52]: println(0.1 + 0.2 == 0.3)

      false
```

Рис. 2.30: Особенность сравнения вещественных чисел

- **Логические операции**

```
[53]: x = true
      y = false

      println(!x)
      println(x && y)
      println(x || y)

      false
      false
      true
```

Рис. 2.31: Пример использования логических операций

4. Привёл несколько своих примеров с операциями над матрицами и векторами: сложение, вычитание, скалярное произведение, транспонирование, умножение на скаляр.

- **Сложение**

```
[54]: a = [1 2; 3 4]
      b = [5 6; 7 8]
      c = a + b
      println(c)

      [6 8; 10 12]
```

Рис. 2.32: Пример сложения матриц

```
[55]: a = [1, 2, 3]
      b = [4, 5, 6]
      c = a + b
      println(c)

      [5, 7, 9]
```

Рис. 2.33: Пример сложения векторов

- **Вычитание**

```
[56]: a = [10 9; 8 7]
      b = [6 5; 4 8]
      c = a - b
      println(c)

      [4 4; 4 -1]
```

Рис. 2.34: Пример вычитания матриц

```
[57]: a = [1, 2, 3]
      b = [4, 5, 6]
      c = b - a
      println(c)

      [3, 3, 3]
```

Рис. 2.35: Пример вычитания векторов

- **Скалярное произведение**

```
[60]: using LinearAlgebra

      a = [1, 2, 3]
      b = [4, 5, 6]
      c = dot(a, b)
      println(c)

      32
```

Рис. 2.36: Пример скалярного произведения векторов

- **Транспонирование**

```
[61]: M = [1 2 3; 4 5 6]
      M_transposed = M'
      println(M_transposed)

      [1 4; 2 5; 3 6]
```

Рис. 2.37: Пример транспонирования матрицы

```
[62]: w = [1, 2, 3]
      w_row = w'
      println(w_row)

      adjoint([1, 2, 3])
```

Рис. 2.38: Пример транспонирования вектора

- **Умножение на скаляр**

```
[63]: a = [1, 2, 3]
      c = 2 * a
      println(c)

      [2, 4, 6]
```

Рис. 2.39: Пример умножения вектора на скаляр

```
[64]: a = [1 2; 3 4]
      c = 2 * a
      println(c)

      [2 4; 6 8]
```

Рис. 2.40: Пример умножения матрицы на скаляр

3 Выводы

В результате выполнения лабораторной работы, я подготовил рабочее пространство и инструментарий для работы с языком программирования Julia, на простейших примерах познакомился с основами синтаксиса Julia.